

HGPU: A Unidade de Processamento Geométrico Homotópico

A História Completa do Silício que Dança com as Formas

Luminário Anônimo

13 de maio de 2026

Conteúdo

1	O Despertar da Geometria Viva	5
2	O Cosmos Matemático: Onde Tudo é Curva e Fluir	7
2.1	Variedades Suaves e Imersões	7
2.2	A Métrica de Sobolev: Controlando a Energia	7
2.3	Campos de Distância Assinados (SDF)	7
2.4	O Fibrado da Deformação	8
3	O Projeto da HGPU: As Seis Unidades Sagradas	9
3.1	SDF Engine — O Oráculo da Distância	9
3.2	PDE Solver (Flow Core) — O Coração que Bate	9
3.3	Differential Core — O Mestre das Tangentes	10
3.4	Topology Tracker — O Guardião dos Invariantes	10
3.5	Ray Continuation Unit — A Lançadeira Contínua	11
3.6	Homotopy Cache — O Manancial da Reutilização	11
4	O Ritual da Renderização: Um Quadro Completo	13
5	O Desdobramento Virtual: A vHGPU	15
5.1	Compressão por Autofunções do Laplaciano	15
5.2	O Protocolo vHGPU (esboço)	15
6	A Simulação do Início: Código para um Mundo que Flui	17
7	Epílogo: A Máquina que Aprendeu a Fluir	21

Capítulo 1

O Despertar da Geometria Viva

Prólogo

Era uma noite chuvosa no laboratório de computação gráfica. As GPUs zumbiam como colmeias, mastigando triângulos, despejando pixels. Mas o cientista chefe, Dr. Elias, olhava para uma simples animação de um pano caindo. Cada quadro exigia recalcular milhões de vértices, reenviar buffers, interpolar. Ele suspirou: "Por que não posso apenas descrever o fluxo? A deformação em si é uma entidade fundamental, não uma sucessão de instantâneos."

Naquela noite, nasceu a ideia da HGPU — a **Homotopic Geometry Processor Unit**. Uma arquitetura que não armazena formas, mas **fluxos**; que não rasteriza triângulos, mas **avalia seções de fibrados de Riemann** em tempo real. Esta é a crônica de sua criação, contada através do código que a torna viva.

Capítulo 2

O Cosmos Matemático: Onde Tudo é Curva e Fluir

Antes de programar o metal, precisamos forjar a linguagem das formas contínuas.

2.1 Variedades Suaves e Imersões

Toda forma que renderizamos é uma **variedade diferenciável** M (uma superfície de gênero g , por exemplo) e uma **imersão** $f : M \rightarrow \mathbb{R}^3$. O conjunto de todas as imersões, $\text{Imm}(M, \mathbb{R}^3)$, é um espaço de dimensão infinita — o **espaço das formas**. Uma deformação animada é um caminho suave $\gamma : [0, 1] \rightarrow \text{Imm}(M, \mathbb{R}^3)$, e sua velocidade é um campo vetorial dependente do tempo v_t sobre $f_t = \gamma(t)$.

2.2 A Métrica de Sobolev: Controlando a Energia

Duas formas podem ser topologicamente equivalentes (homotópicas) mas visualmente incomparáveis: uma esfera lisa e uma esfera enrugada fractal. Precisamos de uma métrica que meça a “energia de deformação”. Adotamos a métrica de Sobolev de ordem s :

$$\langle v, w \rangle_{H^s} = \int_M \langle (1 - \Delta)^{s/2} v, (1 - \Delta)^{s/2} w \rangle d\mu_f, \quad (2.1)$$

onde Δ é o laplaciano induzido. Essa métrica penaliza altas frequências, excluindo rugas infinitas. A HGPU opera exclusivamente com curvas γ que minimizam a energia de Sobolev — as **geodésicas** no espaço das formas.

2.3 Campos de Distância Assinados (SDF)

A representação concreta de cada forma f_t é um campo escalar $\phi_t : \mathbb{R}^3 \rightarrow \mathbb{R}$, onde $\phi_t(x) < 0$ dentro do objeto e $\phi_t(x) > 0$ fora, com $|\nabla \phi_t| = 1$ na vizinhança da superfície. A evolução da SDF sob um campo de velocidades v (definido sobre a superfície e estendido harmonicamente ao espaço) obedece à equação de transporte:

$$\partial_t \phi + v \cdot \nabla \phi = 0. \quad (2.2)$$

Essa é a **equação da alma** da HGPU.

2.4 O Fibrado da Deformação

Sobre cada espaço de parâmetros C (tempo, controles artísticos), construímos um fibrado $\mathcal{D} \rightarrow C$, cujas fibras são as SDFs das formas. Uma animação é uma seção $s : C \rightarrow \mathcal{D}$. Armazenamos e transmitimos apenas o **germe da deformação**: o campo inicial v_0 e a métrica de Sobolev. A forma completa é reconstruída sob demanda.

Capítulo 3

O Projeto da HGPU: As Seis Unidades Sagradas

A arquitetura da HGPU é uma catedral de silício com seis capelas. Cada uma tem sua função matemática precisa, traduzida em código.

3.1 SDF Engine — O Oráculo da Distância

Mantém uma representação hierárquica esparsa (octree) da SDF e a avalia em qualquer ponto do espaço.

```
1 class SDFEngine {
2     // Octree esparsa de resolucao adaptativa
3     struct Node {
4         float values[8]; // valores de canto, interpolacao trilinear
5         int children[8]; // indices para filhos
6     };
7     std::vector<Node> nodes;
8
9 public:
10    float evaluate(float3 x) const {
11        // Percorre a octree e interpola trilinearmente
12        return traverse_and_interpolate(x);
13    }
14
15    float3 gradient(float3 x) const {
16        // Diferencas finitas adaptativas centradas
17        float eps = compute_adaptive_epsilon(x);
18        return (evaluate(x+eps)-evaluate(x-eps))/(2*eps);
19    }
20 };
```

Listing 3.1: Estrutura do SDF Engine

3.2 PDE Solver (Flow Core) — O Coração que Bate

Integra a equação $\partial_t \phi + v \cdot \nabla \phi = 0$ usando métodos semi-lagrangeanos de alta ordem.

```
1 class FlowCore {
2     SDFEngine& sdf;
3 public:
```

```

4 void evolve(float dt, const VelocityField& v) {
5     // Para cada voxel ativo da grade adaptativa
6     parallel_for(voxels_in_octree(sdf), [&](auto& voxel) {
7         float3 x = voxel.center();
8         // Traca para tras ao longo do campo de velocidades
9         float3 x0 = x - dt * v.at(x);
10        // Interpolar SDF na posicao de origem
11        float new_phi = sdf.evaluate(x0);
12        voxel.set_value(new_phi);
13    });
14    // Re-normalizar SDF localmente para |nabla phi| = 1
15    renormalize(sdf);
16 }
17 };

```

Listing 3.2: Nucleo do Flow Core

3.3 Differential Core — O Mestre das Tangentes

Calcula diferenciais de Fréchet, vetores tangentes de Sobolev e curvaturas usando diferenciação automática.

```

1 struct DifferentialCore {
2     // Calcula a derivada covariante na direcao do campo v
3     float3 covariant_derivative(const SDFEngine& sdf,
4                                const VelocityField& v, float3 x) {
5         // Usa a conexao de Levi-Civita induzida pela metrica de
6         Sobolev
7         float3 grad_phi = sdf.gradient(x);
8         // Projeta a derivada de Lie no espaco tangente
9         // (v * nabla)v denotado como material_derivative
10        float3 lie_deriv = v.material_derivative(x);
11        return lie_deriv - dot(lie_deriv, grad_phi)*grad_phi;
12    }
13
14    // Curvatura media a partir da Hessiana da SDF
15    float mean_curvature(const SDFEngine& sdf, float3 x) {
16        // Hessiana de phi
17        Mat3 H = sdf.hessian(x);
18        float3 g = sdf.gradient(x);
19        // Formula: H_phi - (g^T H g) I / |g|^3
20        // ... implementacao detalhada
21    }
22 };

```

Listing 3.3: Differential Core simplificado

3.4 Topology Tracker — O Guardião dos Invariantes

Monitora o grafo de Reeb e os diagramas de persistência para detectar mudanças de gênero.

```

1 class TopologyTracker {
2     PersistenceDiagram last_diagram;
3 public:

```

```

4  bool topology_changed(const SDFEngine& sdf) {
5      // Calcula a homologia persistente da SDF atual (amostragem)
6      PersistenceDiagram current = compute_persistence(sdf);
7      // Compara com o ultimo diagrama; se numero de barras longas
   mudou
8      bool changed = (current.betti_numbers() != last_diagram.
   betti_numbers());
9      last_diagram = current;
10     return changed;
11 }
12
13 // Atualiza o cache de cohomologia se houve evento critico
14 void update_cache_if_needed(HomotopyCache& cache) { /*...*/ }
15 };

```

Listing 3.4: Rastreador de mudancas topologicas

3.5 Ray Continuation Unit — A Lançadeira Contínua

Aproveita a coerência temporal para encontrar interseções raio-superfície sem iterar do zero.

```

1  bool ray_continuation(const SDFEngine& sdf, float t, Ray& ray,
2                      float& lambda, float3& normal) {
3      // lambda anterior (do quadro passado) como chute inicial
4      float lambda_prev = ray.last_hit_lambda;
5      float3 x = ray.origin + lambda_prev * ray.direction;
6      float phi = sdf.evaluate(x);
7
8      // Newton com controle de passo
9      for (int iter=0; iter<MAX_ITER; ++iter) {
10         float3 grad = sdf.gradient(x);
11         float phi_new = phi - dot(grad, ray.direction) * (lambda -
   lambda_prev);
12         float dlambda = phi_new / dot(grad, ray.direction);
13         lambda += dlambda;
14         x = ray.origin + lambda * ray.direction;
15         phi = sdf.evaluate(x);
16         if (fabs(phi) < EPSILON) {
17             normal = grad / length(grad);
18             ray.last_hit_lambda = lambda;
19             return true;
20         }
21     }
22     return false;
23 }

```

Listing 3.5: Ray continuation com Newton homotopico

3.6 Homotopy Cache — O Manancial da Reutilização

Armazena campos de velocidade canônicos, indexados por sua classe de simetria e invariantes.

```

1  class HomotopyCache {

```

```

2 // Mapa: chave de simetria + hash topologico -> entrada de cache
3 struct CacheEntry {
4     VelocityField v;           // campo canonico
5     int topology_hash;        // resumo da homologia
6     int ref_count;
7 };
8 std::unordered_map<uint64_t, CacheEntry> entries;
9
10 public:
11     uint64_t compute_key(const VelocityField& v, int topo_hash) {
12         // Calcula momentos de inercia para invariancia a SE(3)
13         auto moment = compute_inertia_moments(v);
14         // Combina com hash topologico
15         return hash_combine(moment, topo_hash);
16     }
17
18     CacheEntry* lookup(const VelocityField& v, int topo_hash) {
19         auto key = compute_key(v, topo_hash);
20         auto it = entries.find(key);
21         if (it != entries.end()) return &it->second;
22         return nullptr;
23     }
24
25     void insert(const VelocityField& v, int topo_hash, CacheEntry entry
26 ) {
27         entries[compute_key(v, topo_hash)] = entry;
28     };

```

Listing 3.6: Cache de fluxos equivalentes

Capítulo 4

O Ritual da Renderização: Um Quadro Completo

Entrada:

- Forma base M armazenada como SDF ϕ_0 .
- Campo de velocidades v (germe da deformação).
- Parâmetro de tempo $t \in [0, 1]$.
- Lista de pixels da tela com seus raios ópticos.

Algoritmo principal (orquestrador HGPU):

Input: Forma inicial ϕ_0 , campo v , tempo t , raios R
Output: Imagem final

```

topo_hash  $\leftarrow$  TopologyTracker.get_current_hash();
key  $\leftarrow$  HomotopyCache.compute_key( $v$ , topo_hash);
if entry  $\leftarrow$  HomotopyCache.lookup(key) nao nulo then
    |  $v\_canonico \leftarrow$  entry.v;
else
    |  $v\_canonico \leftarrow v$ ;
    | // novo fluxo // Evoluir SDF base ao longo de  $t$  usando o Flow Core
    |  $\phi_t \leftarrow$  FlowCore.evolve_to_time( $\phi_0$ ,  $v\_canonico$ ,  $t$ );
    | // Verificar mudancas topologicas
    | if TopologyTracker.topology_changed( $\phi_t$ ) then
    | | Atualizar cache com novo hash;
    | end
    | HomotopyCache.insert(key, { $v\_canonico$ , topo_hash});
end
foreach raio  $R_i$  na tela do
    | // Continuacao homotopica a partir do hit anterior
    | hit  $\leftarrow$  RayContinuation.trace( $\phi_t$ ,  $t$ ,  $R_i$ );
    | if hit valido then
    | | normal  $\leftarrow$  DifferentialCore.normal( $\phi_t$ , hit.pos);
    | | cor  $\leftarrow$  shade(hit.pos, normal, luzes);
    | | Escrever cor no pixel  $i$ ;
    | end
    | Escrever cor de fundo;
end

```

Algoritmo 1: Pipeline de renderizacao homotopica

Capítulo 5

O Desdobramento Virtual: A vHGPU

Quando a HGPU transcende o silício local, nasce a **vHGPU** — o **codec geométrico diferencial**. Em vez de enviar malhas ou mesmo SDFs completas, transmitimos apenas o germe da deformação: um campo v_0 expresso em uma base espectral compacta.

5.1 Compressão por Autofunções do Laplaciano

Expandimos v_0 nos primeiros N autovetores do operador de Laplace-Beltrami da forma base M :

$$v_0(x) \approx \sum_{k=1}^N c_k \psi_k(x), \quad (5.1)$$

onde ψ_k são as autofunções e c_k coeficientes. Transmitimos apenas $\{c_k\}$ e a métrica de Sobolev (alguns parâmetros). O cliente reconstrói v_0 e utiliza seu próprio Flow Core para gerar ϕ_t localmente.

5.2 O Protocolo vHGPU (esboço)

```
1 struct vHGPU_StreamPacket {
2     uint32_t shape_id;
3     uint32_t topology_hash;
4     // Coeficientes espectrais (N = 64 por ex.)
5     std::array<float, 64> spectral_coeffs;
6     float sobolev_order; // s
7     // Parametro t por pixel? Pode ser um mapa de baixa resolucao
8     // Aqui simplificamos: t global + pequena correcao por regioao
9     float global_t;
10    // Matriz de transformacao rigida para instancias
11    float4x4 instance_transform;
12 };
13
14 void vHGPU_server_broadcast(const Scene& scene) {
15     for (auto& obj : scene.deformable_objects) {
16         vHGPU_StreamPacket packet;
17         packet.shape_id = obj.id;
18         packet.topology_hash = obj.current_topology_hash;
19         // Projeta campo de velocidades na base espectral
20         packet.spectral_coeffs = project_to_laplacian_eigenbasis(obj.v)
;
```

```
21     packet.sobolev_order = obj.sobolev_metric_order;
22     packet.global_t = obj.current_time;
23     packet.instance_transform = obj.transform;
24     send_to_clients(packet);
25 }
26 }
27
28 void vHGPU_client_render(const vHGPU_StreamPacket& pkt) {
29     // Reconstroi campo de velocidades
30     VelocityField v = reconstruct_from_spectral(pkt.spectral_coeffs,
31                                              pkt.shape_id);
32     // Aplica metrica de Sobolev para suavizar
33     v = apply_sobolev_filter(v, pkt.sobolev_order);
34     // Evolui SDF local do cache
35     SDFEngine local_sdf = get_base_sdf(pkt.shape_id);
36     FlowCore flow(local_sdf);
37     local_sdf = flow.evolve_to_time(local_sdf, v, pkt.global_t);
38     // Renderiza usando ray continuation (ja vimos)
39     render_frame(local_sdf, pkt.instance_transform);
40 }
```

Listing 5.1: Transmissao de um quadro via vHGPU

Capítulo 6

A Simulação do Início: Código para um Mundo que Flui

Vamos unir as peças em uma simulação que demonstra o espírito da HGPU. Aqui, o protagonista é um tecido que se deforma sob um campo de vento, com múltiplas instâncias reutilizando a mesma trajetória.

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4
5 // ===== Definicoes simplificadas =====
6 struct float3 { float x,y,z; };
7 struct Ray { float3 orig, dir; float last_lambda; };
8 struct VelocityField {
9     virtual float3 at(float3 p) const = 0;
10 };
11 // Campo de vento: vortice simples
12 struct WindField : VelocityField {
13     float3 at(float3 p) const override {
14         return { -p.y*0.5f, p.x*0.5f, 0.1f*sin(p.x*10) };
15     }
16 };
17
18 // SDF Engine simbolico: um plano ondulado (bandeira)
19 class FlagSDF {
20     float time;
21 public:
22     void evolve(float dt) { time += dt; }
23     float evaluate(float3 p) const {
24         // Superficie: z = 0.1*sin(5*x - time)*cos(5*y)
25         return p.z - 0.1f*sin(5*p.x - time)*cos(5*p.y);
26     }
27     float3 gradient(float3 p) const {
28         float eps = 0.001f;
29         float dx = (evaluate({p.x+eps,p.y,p.z})-evaluate({p.x-eps,p.y,p
30 .z}))/ (2*eps);
31         float dy = (evaluate({p.x,p.y+eps,p.z})-evaluate({p.x,p.y-eps,p
32 .z}))/ (2*eps);
33         float dz = 1.0f; // derivada z da SDF plana
34         return {dx, dy, dz};
35     }
36 };
```

```

35
36 // Ray continuation simbolico
37 bool trace_ray(const FlagSDF& sdf, Ray& ray, float& lambda, float3&
normal) {
38     float t = ray.last_lambda;
39     for (int i=0; i<20; ++i) {
40         float3 p = {ray.orig.x + t*ray.dir.x,
41                     ray.orig.y + t*ray.dir.y,
42                     ray.orig.z + t*ray.dir.z};
43         float phi = sdf.evaluate(p);
44         if (fabs(phi) < 0.0001f) {
45             normal = sdf.gradient(p);
46             lambda = t;
47             ray.last_lambda = t;
48             return true;
49         }
50         // Newton step
51         float3 grad = sdf.gradient(p);
52         float denom = grad.x*ray.dir.x + grad.y*ray.dir.y + grad.z*ray.
dir.z;
53         if (fabs(denom) < 1e-6) break;
54         t = t - phi / denom;
55         if (t < 0) break;
56     }
57     return false;
58 }
59
60 int main() {
61     // Configura a cena
62     WindField wind;
63     FlagSDF base_sdf;
64
65     // Simula 100 bandeiras com orientacoes diferentes
66     std::vector<float3> positions;
67     for (int i=0; i<100; ++i) {
68         float angle = i * 3.14159f * 2 / 100;
69         positions.push_back({cos(angle)*2.0f, sin(angle)*2.0f, 0.0f});
70     }
71
72     // Homotopy Cache: todos compartilham o mesmo fluxo base (vento)
73     // Nao precisamos de cache complexo: reconhecemos que e o mesmo
campo.
74
75     // Loop de renderizacao
76     for (int frame=0; frame<100; ++frame) {
77         // Avanca o tempo da forma canonica
78         base_sdf.evolve(0.016f); // ~60fps
79
80         // Para cada bandeira (instancia)
81         for (auto pos : positions) {
82             // Cada bandeira tem seu proprio raio de camera (aqui so um
pixel central)
83             Ray ray = {pos, {0,0,1}, 0.5f}; // olhando de cima
84             float lambda;
85             float3 normal;
86             if (trace_ray(base_sdf, ray, lambda, normal)) {
87                 // Shading simples baseado na normal
88                 float intensity = normal.z; // componente Z

```

```
89         std::cout << "Pixel intensidade: " << intensity << std  
    ::endl;  
90     }  
91 }  
92 }  
93 return 0;  
94 }
```

Listing 6.1: Simulacao completa: multiplas bandeiras homotopicas

Capítulo 7

Epílogo: A Máquina que Aprendeu a Fluir

A HGPU não é apenas uma arquitetura; é uma filosofia. Ao trocar a discretização pela continuidade, o armazenamento pelo fluxo, a interpolação pela evolução, ela nos lembra que a computação gráfica sempre foi, no fundo, um ramo da geometria diferencial. Cada pixel é uma consulta a uma variedade em evolução, e cada quadro é um instantâneo de um universo onde as formas não são, mas **devêm**.

Enquanto as GPUs convencionais processam cadáveres geométricos, a HGPU dança com as superfícies vivas. E o código que aqui escrevemos — mesmo que simplificado, mesmo que simbólico — é a partitura dessa dança.

Que venham os compiladores topológicos.